

Padrões de repetição, estruturas de dados sequenciais

Programação A

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação – DCOMP

Padrões de Iteração

Sequências de Dados

Mutabilidade e Imutabilidade

Referências e Aliasing

Padrões Algorítmicos de Repetição

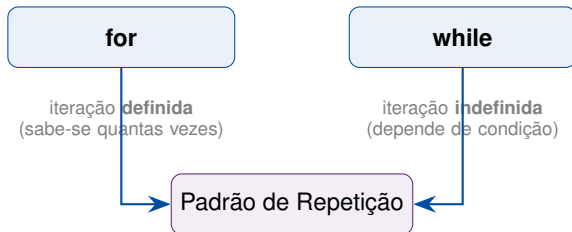
Matrizes (Listas de Listas)

Resumo e Exercícios

Padrões de Iteração

Ideia central

“Muitos problemas computacionais envolvem a repetição de um padrão. A iteração é o mecanismo que permite expressar essa repetição de forma concisa e controlada.”

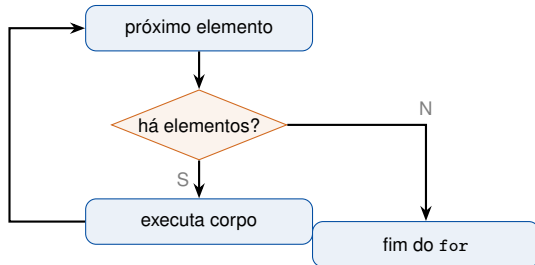


Estrutura geral

O `for` percorre cada elemento de uma **sequência** (lista, range, string, tupla).

```
1 for variavel in sequencia:  
2     # corpo: executado uma  
3     # vez para cada elemento  
4     ...
```

```
1 >>> for x in [10, 20, 30]:  
2     ...     print(x * 2)  
3 20  
4 40  
5 60
```



Gerando sequências numéricas

`range` cria uma sequência de inteiros **sem armazenar todos na memória** (avaliação preguiçosa).

Chamada	Resultado	Descrição
<code>range(5)</code>	0, 1, 2, 3, 4	início=0, passo=1
<code>range(2, 7)</code>	2, 3, 4, 5, 6	início=2, fim exclusivo=7
<code>range(0, 10, 2)</code>	0, 2, 4, 6, 8	passo=2
<code>range(5, 0, -1)</code>	5, 4, 3, 2, 1	contagem regressiva

```
1 >>> for i in range(1, 6):
2     ...     print(i ** 2, end=" ")
3 1 4 9 16 25
```

Atenção

O limite superior é **exclusivo**: `range(5)` gera de 0 a 4 (não inclui 5).

Quando precisamos do índice e do valor

```
1 >>> frutas = ["manga", "caju", "goiaba"]
2 >>> for i, fruta in enumerate(frutas):
3 ...     print(f"{i}: {fruta}")
4 0: manga
5 1: caju
6 2: goiaba
```



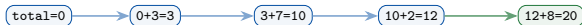
Dica

Evite o padrão `for i in range(len(lista))` quando precisa apenas iterar. Use `for item in lista` OU `enumerate`.

Problema

Calcule a soma dos elementos de uma lista.

```
1 def soma(lista):  
2     total = 0                # inicializa acumulador  
3     for x in lista:  
4         total += x          # atualiza acumulador  
5     return total            # retorna resultado  
6  
7 >>> soma([3, 7, 2, 8])  
8 20
```



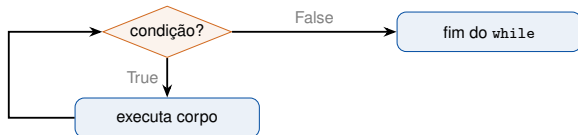
Padrão geral

Inicializar → **Iterar e atualizar** → **Retornar**. Este é um dos padrões mais recorrentes em programação.

Estrutura geral

O `while` repete enquanto uma **condição** for verdadeira. Usado quando não sabemos de antemão quantas iterações serão necessárias.

```
1 n = 1
2 while n <= 5:
3     print(n, end=" ")
4     n += 1
5 # Saída: 1 2 3 4 5
```



Cuidado: laço infinito

Se a condição **nunca** se tornar falsa, o programa não termina. Sempre garanta que algo no corpo do laço muda a condição.

Exemplo: Adivinhe o Número



```
1 import random
2
3 secreto = random.randint(1, 100)
4 palpite = 0
5
6 while palpite != secreto:
7     palpite = int(input("Seu palpite: "))
8     if palpite < secreto:
9         print("Muito baixo!")
10    elif palpite > secreto:
11        print("Muito alto!")
12
13 print("Acertou!")
```

Observação

Aqui o número de iterações é **desconhecido** — depende da entrada do usuário. Esse é o caso clássico para `while`.

: sai do laço

```
for n in range(2, 100):  
    if eh_primo(n):  
        print(n)  
        break  
# imprime 2 (primeiro primo)
```

break

encerra o laço
imediatamente

: pula para próxima iteração

```
for i in range(10):  
    if i % 2 == 0:  
        continue  
    print(i, end=" ")  
# Saída: 1 3 5 7 9
```

continue

pula para a próxima
iteração do laço

Quando usar `for` vs. `while`?



<code>for</code>	<code>while</code>
Quantidade de iterações conhecida	Quantidade de iterações desconhecida
Percorrer uma sequência	Repetir até uma condição mudar
<code>for x in lista:</code>	<code>while not encontrou:</code>
Processar cada item	Buscar, convergir, ler entrada

Regra prática

Se você sabe *sobre o que* iterar (uma coleção ou um range), use `for`. Se você sabe *quando parar* (uma condição), use `while`.

Sequências de Dados

O que é uma sequência?

Uma **sequência** é uma coleção **ordenada** de elementos, acessíveis por **índice** (posição).

Listas

`[1, 2, 3]`

Ranges

`range(5)`

Strings

`"UFS"`

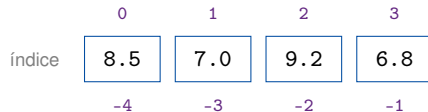
Tuplas

`(1, 2, 3)`

Todas suportam: indexação `[i]`, fatiamento `[a:b]`, `len()`, `in`, iteração com `for`

Criação e acesso

```
1 >>> notas = [8.5, 7.0, 9.2, 6.8]
2 >>> notas[0]
3 8.5
4 >>> notas[-1]
5 6.8
6 >>> len(notas)
7 4
```



Índices negativos

-1 é o último, -2 o penúltimo, etc.

Operação	Exemplo	Resultado
Adicionar ao final	<code>lst.append(4)</code>	<code>[1,2,3,4]</code>
Inserir na posição	<code>lst.insert(1, 99)</code>	<code>[1,99,2,3]</code>
Remover por valor	<code>lst.remove(2)</code>	<code>[1,3]</code>
Remover por índice	<code>lst.pop(0)</code>	retorna 1
Ordenar	<code>lst.sort()</code>	ordena in-place
Inverter	<code>lst.reverse()</code>	inverte in-place
Concatenar	<code>[1,2] + [3,4]</code>	<code>[1,2,3,4]</code>
Repetir	<code>[0] * 3</code>	<code>[0,0,0]</code>
Pertinência	<code>2 in [1,2,3]</code>	<code>True</code>

Nota

Métodos como `append`, `sort` e `reverse` **modificam** a lista original (operações *in-place*) e retornam `None`.


```
1 >>> s = "Sergipe"
2 >>> s[0]
3 'S'
4 >>> s[-1]
5 'e'
6 >>> len(s)
7 7
8 >>> "rg" in s
9 True
```

Métodos úteis

```
>>> "ufs".upper()
'UFS'
>>> "  oi  ".strip()
'oi'
>>> "a,b,c".split(",")
['a', 'b', 'c']
>>> "-".join(["x","y","z"])
'x-y-z'
>>> "cafe".replace("e","é")
'café'
```

0	1	2	3	4	5	6
S	e	r	g	i	p	e

Sequências imutáveis

Tuplas são como listas, mas **não podem ser modificadas** após a criação.

```
1 >>> coord = (10.5, -37.0)
2 >>> coord[0]
3 10.5
4 >>> coord[1] = 20      # ERRO! Tuplas são imutáveis
5 TypeError: 'tuple' object does not support item assignment
6
7 >>> ponto = (3, 4)
8 >>> x, y = ponto      # desempacotamento
9 >>> x
10 3
```

Quando usar tuplas?

- Dados que não devem mudar: coordenadas, datas, registros fixos.
- Retorno múltiplo de funções: `return (min, max)`.
- Chaves de dicionários (tuplas são *hashable*).

Sintaxe geral

sequencia[inicio:fim:passo] — início incluso, fim **excluso**.

```
1 >>> nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 >>> nums[2:5]           # indices 2, 3, 4
3 [2, 3, 4]
4 >>> nums[:3]            # do inicio ate 2
5 [0, 1, 2]
6 >>> nums[7:]           # do 7 ate o final
7 [7, 8, 9]
8 >>> nums[::2]           # passo 2 (pares de indice)
9 [0, 2, 4, 6, 8]
10 >>> nums[::-1]         # inverter
11 [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

```
1 # Strings
2 >>> "Universidade"[0:5]
3 'Unive'
4 >>> "Python"[::-1]
5 'nohtyP'
6
7 # Tuplas
8 >>> t = (10, 20, 30, 40, 50)
9 >>> t[1:4]
10 (20, 30, 40)
11
12 # Range (retorna range)
13 >>> list(range(10)[:3])
14 [0, 3, 6, 9]
```

Forma concisa de criar listas

```
1 >>> [x ** 2 for x in range(1, 6)]
2 [1, 4, 9, 16, 25]
3
4 >>> [x for x in range(20) if x % 3 == 0]
5 [0, 3, 6, 9, 12, 15, 18]
6
7 >>> [c.upper() for c in "sergipe" if c != 'e']
8 ['S', 'R', 'G', 'I', 'P']
```

[<expressão> for <var> in <sequência> if <condição>]

Equivalência

[f(x) for x in seq if p(x)] equivale ao pipeline `map` + `filter` da aula anterior.

Mutabilidade e Imutabilidade

Imutáveis

`int, float, bool`
`str, tuple`

Não podem ser alterados
após a criação.
Qualquer “modificação” cria
um **novo objeto**.

Mutáveis

`list, dict`
`set`

Podem ser alterados
in-place.
O **mesmo objeto** muda
de conteúdo.

Strings são imutáveis

```
>>> s = "gato"
>>> s[0] = "p"
TypeError: 'str' object does
  not support item assignment

# Solucao: criar nova string
>>> s = "p" + s[1:]
>>> s
'pato'
```

Listas são mutáveis

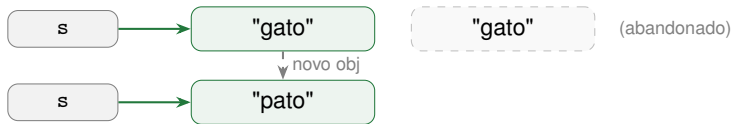
```
>>> lst = [1, 2, 3]
>>> lst[0] = 99
>>> lst
[99, 2, 3]    # mesma lista!

>>> id(lst)    # mesmo id
140234567890
```

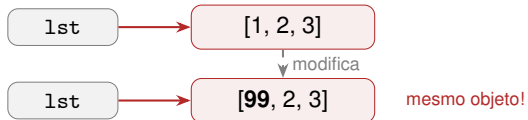
Função `id()`

A função `id()` retorna o identificador único de um objeto na memória. Para imutáveis, operações “de modificação” geram um **novo** `id`. Para mutáveis, o `id` permanece o **mesmo**.

Imutável (`str`)



Mutável (`list`)

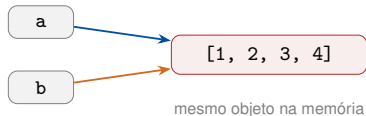


Referências e Aliasing

Modelo mental

Em Python, variáveis **não contêm** os dados diretamente. Elas guardam **referências** (“setas”) para objetos na memória.

```
1 >>> a = [1, 2, 3]
2 >>> b = a           # aliasing!
3 >>> b.append(4)
4 >>> a
5 [1, 2, 3, 4]        # a também mudou!
```



Aliasing

Quando duas variáveis apontam para o **mesmo objeto mutável**, qualquer modificação via uma delas é visível pela outra. Isso é chamado de **aliasing**.

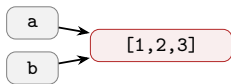
Aliasing (mesma referência)

```
>>> a = [1, 2, 3]
>>> b = a
>>> a is b
True
>>> id(a) == id(b)
True
```

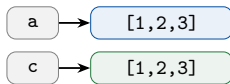
Cópia (novo objeto)

```
>>> a = [1, 2, 3]
>>> c = a[:]      # copia rasa
>>> c = list(a)   # tambem copia
>>> a is c
False
>>> c.append(4)
>>> a
[1, 2, 3] # nao afeta a!
```

Aliasing



Cópia



Efeitos colaterais com listas

Quando passamos uma lista para uma função, a função recebe a **referência**. Modificações dentro da função afetam a lista original.

```
1 def adiciona_zero(lst):
2     lst.append(0)      # modifica a lista original!
3
4 >>> minha_lista = [1, 2, 3]
5 >>> adiciona_zero(minha_lista)
6 >>> minha_lista
7 [1, 2, 3, 0]          # foi modificada!
```

Versão segura (sem efeito colateral)

```
def adiciona_zero_segura(lst):
    return lst + [0]    # cria nova lista
```

Igualdade (==) vs. Identidade (is)

Operador	Pergunta	Verifica
==	Têm o mesmo conteúdo ?	Igualdade de valor
is	São o mesmo objeto ?	Identidade (mesmo id)

```
1 >>> a = [1, 2, 3]
2 >>> b = [1, 2, 3]
3 >>> a == b
4 True                # mesmo conteúdo
5 >>> a is b
6 False              # objetos diferentes na memória!
7
8 >>> c = a
9 >>> a is c
10 True              # aliasing: mesmo objeto
```

Padrões Algorítmicos de Repetição

Acumulador

somar, produzir, concatenar

Contagem

contar ocorrências

Busca/Pesquisa

encontrar máximo, mínimo, posição

Filtragem

selecionar com critério

Transformação

produzir nova lista transformada

Problema

Conte quantos números negativos existem em uma lista.

```
1 def contar_negativos(lista):
2     contagem = 0
3     for x in lista:
4         if x < 0:
5             contagem += 1
6     return contagem
7
8 >>> contar_negativos([3, -1, 4, -5, -2, 8])
9 3
```

Estrutura

Inicializar contador → **Iterar e testar condição** → **Incrementar se verdadeiro** → **Retornar**.

Problema

Encontre o maior elemento de uma lista (sem usar `max`).

```
1 def maior(lista):  
2     maximo = lista[0]  
3     for x in lista[1:]:  
4         if x > maximo:  
5             maximo = x  
6     return maximo  
7  
8 >>> maior([3, 7, 2, 9, 1])  
9 9
```

Estrutura

Inicializar candidato → Iterar e comparar → Atualizar se melhor → Retornar.

Problema

Dada uma lista de notas, retorne apenas as notas acima de 7.0.

Com laço explícito

```
def aprovados(notas):  
    resultado = []  
    for n in notas:  
        if n >= 7.0:  
            resultado.append(n)  
    return resultado
```

Com list comprehension

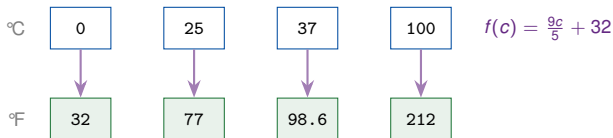
```
def aprovados(notas):  
    return [n for n in notas  
            if n >= 7.0]
```

```
>>> aprovados([8.5, 6.0, 9.2, 5.5, 7.0])  
[8.5, 9.2, 7.0]
```

Problema

Converter temperaturas de Celsius para Fahrenheit.

```
1 def celsius_para_fahrenheit(temperaturas):  
2     return [c * 9/5 + 32 for c in temperaturas]  
3  
4 >>> celsius_para_fahrenheit([0, 25, 37, 100])  
5 [32.0, 77.0, 98.6, 212.0]
```



Matrizes (Listas de Listas)

Representação

Em Python, uma matriz $m \times n$ é uma lista de m listas, cada uma com n elementos.

```
1  # Matriz 3x3
2  M = [
3      [1, 2, 3],
4      [4, 5, 6],
5      [7, 8, 9]
6  ]
7
8  >>> M[0]           # linha 0
9  [1, 2, 3]
10 >>> M[1][2]        # linha 1, col
11                     2
12 6
```

	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6
[2]	7	8	9

`M[linha][coluna]`

Acesso: primeiro a **linha**, depois a **coluna**.

Matriz de zeros ($m \times n$)

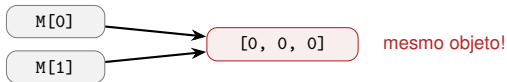
```
def zeros(m, n):  
    return [[0] * n  
            for _ in range(m)]
```

```
>>> zeros(2, 3)  
[[0, 0, 0],  
 [0, 0, 0]]
```

Armadilha com *

```
# ERRADO: aliasing!  
M = [[0] * 3] * 2  
M[0][0] = 1  
# M == [[1, 0, 0], [1, 0, 0]]  
# Ambas as linhas são  
# o MESMO objeto!
```

Armadilha: `[[0]*3]*2`



```
1 M = [[1, 2, 3],
2      [4, 5, 6],
3      [7, 8, 9]]
4
5 # Percorrer linha a linha
6 for linha in M:
7     for elem in linha:
8         print(elem, end=" ")
9     print()
10
11 # Com indices
12 for i in range(len(M)):
13     for j in range(len(M[0])):
14         print(f"M[{i}][{j}]={M[i][j]}", end=" ")
15     print()
```


Problema

Dadas duas matrizes A e B de mesma dimensão, calcule $C = A + B$.

```
1 def soma_matrizes(A, B):
2     m = len(A)
3     n = len(A[0])
4     C = [[0] * n for _ in range(m)]
5     for i in range(m):
6         for j in range(n):
7             C[i][j] = A[i][j] + B[i][j]
8     return C
```

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Exemplo: Transposta de uma Matriz

```
1 def transposta(M):
2     m = len(M)
3     n = len(M[0])
4     T = [[0] * m for _ in range(n)]
5     for i in range(m):
6         for j in range(n):
7             T[j][i] = M[i][j]
8     return T
9
10 >>> transposta([[1, 2, 3],
11 ...             [4, 5, 6]])
12 [[1, 4], [2, 5], [3, 6]]
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \xrightarrow{T} \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Resumo e Exercícios

Conceito	Ponto-chave
<code>for</code>	Iteração definida sobre sequências
<code>while</code>	Iteração indefinida (baseada em condição)
Listas	Mutáveis, indexáveis, fatiáveis
Strings / Tuplas	Imutáveis, indexáveis, fatiáveis
<code>range</code>	Sequência numérica preguiçosa
Slicing	<code>seq[a:b:p]</code> — cria nova sequência
Referências	Variáveis guardam referências a objetos
Aliasing	Duas variáveis → mesmo objeto mutável
<code>==</code> VS <code>is</code>	Igualdade de valor vs. identidade
Matrizes	Listas de listas; laço duplo para percorrer

Problema

Dados dois vetores $\vec{u}$ e $\vec{v}$ (listas de mesmo tamanho), calcule o produto escalar:

$$\vec{u} \cdot \vec{v} = \sum_{i=0}^{n-1} u_i \times v_i$$

Problema

Dados dois vetores $\vec{u}$ e $\vec{v}$ (listas de mesmo tamanho), calcule o produto escalar:

$$\vec{u} \cdot \vec{v} = \sum_{i=0}^{n-1} u_i \times v_i$$

```
1 def produto_escalar(u, v):
2     total = 0
3     for i in range(len(u)):
4         total += u[i] * v[i]
5     return total
6
7 >>> produto_escalar([1, 2, 3], [4, 5, 6])
8 32    # 1*4 + 2*5 + 3*6
```

Problema

Dados dois vetores $\vec{u}$ e $\vec{v}$ (listas de mesmo tamanho), calcule o produto escalar:

$$\vec{u} \cdot \vec{v} = \sum_{i=0}^{n-1} u_i \times v_i$$

```
1 def produto_escalar(u, v):
2     total = 0
3     for i in range(len(u)):
4         total += u[i] * v[i]
5     return total
6
7 >>> produto_escalar([1, 2, 3], [4, 5, 6])
8 32    # 1*4 + 2*5 + 3*6
```

Problema

Escreva uma função que verifica se uma string é um palíndromo (lê-se igual de trás para frente), ignorando espaços e maiúsculas.

Problema

Escreva uma função que verifica se uma string é um palíndromo (lê-se igual de trás para frente), ignorando espaços e maiúsculas.

```
1 def eh_palindromo(s):
2     s = s.lower().replace(" ", "")
3     return s == s[::-1]
4
5 >>> eh_palindromo("Ana")
6 True
7 >>> eh_palindromo("Socorram me subi no onibus em Marrocos")
8 True
```

Conceitos usados

Métodos de string (`lower`, `replace`), fatiamento reverso (`[::-1]`), comparação de igualdade, e o fato de que strings são **imutáveis** (cada operação cria uma nova string).

Exercício 3 — Multiplicação de Matrizes



Desafio

Implemente a multiplicação de matrizes $C = A \times B$, onde $C_{ij} = \sum_k A_{ik} \times B_{kj}$.

Desafio

Implemente a multiplicação de matrizes $C = A \times B$, onde $C_{ij} = \sum_k A_{ik} \times B_{kj}$.

```
1 def mult_matrizes(A, B):
2     m = len(A)          # linhas de A
3     n = len(B[0])       # colunas de B
4     p = len(B)          # colunas de A / linhas de B
5     C = [[0] * n for _ in range(m)]
6     for i in range(m):
7         for j in range(n):
8             for k in range(p):
9                 C[i][j] += A[i][k] * B[k][j]
10    return C
11
12 >>> mult_matrizes([[1,2],[3,4]], [[5,6],[7,8]])
13 [[19, 22], [43, 50]]
```